CSE 201 CT1 + (CT2/CT3) + CT4

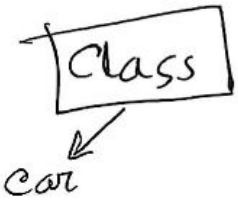
Object Oriented Programming I

Defference off structure programming and OOP

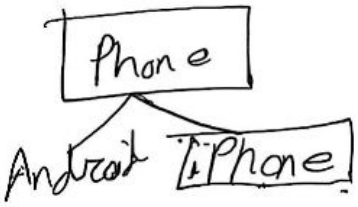
4 ta CT 1st and 4 mandatory

26/7/2026

Class
Object







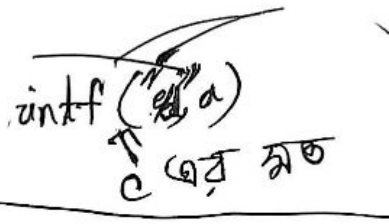


class → iphone 13
object →

object না
(সবার কাছে প্রকাশ করা যাবে না এমন iphone)

IPhone

Object



কপি

না

Page 15 আমার না

28/6/2026

Lecture 1

JRE, JDK এর difference Next দিন বলব।

Hello world print এর প্রোগ্রাম ৩ রকম

```
public class HelloWorldExample
{
    public static void main(String args[])
    {
        System.out.println("Hello World");
    }
}
```

```
public class HelloWorldExample
{
    public static void main(String a[])
    {
        System.out.println("Hello World");
    }
}
```

```
lic class HelloWorldExample
    public static void main(String args[])
    {
        System.out.println("Hello World");
    }
```



```
public class Hi
{
    public static void main (String Cat[])
    {
        System.out.println ("Utsob");
    }
}
```

```
public class Sum
{
    public static void main (String args[])
    {
        int a = 5;
        int b = 10;
        System.out.println(a+b);
    }
}
```

2 + 2 = "4"

"Hello" + "World" = "HelloWorld"

c = 10

↓

"Sum " + c = "Sum 10"

System.out.printf ("%d", a)

c এর মত

System.out.println → কোডে new line add করে

System.out.print → new line add করে না

C তে function
Java তে Method

Java তে if_else ঠিক ~~আগের~~ C এর মতো
" " for loop C এর মতই

24/1/2026

Data type দুই প্রকার।
1. Primitive
2. NON - Premitive / User defined

Class or refarese (two types
→ Textual (String)
→ All classes that declare by yourself

Type Casting দুই প্রকার
1. Premitive Casting: Widening and Narrowing
2. Object Casting: Upcasting and Down casting

1. Widening Casting
Order: byte → short → int → long → float → double

```
int a = 10
double b = a
system.out.print(b)  → output: 10.0
```

↑ Auto casting (বড় তে ছোটটি অর্থাৎ ছোট যায়)

2. Narrowing Casting (Data loss হয়)

float a = 9.75
int b = (int) a

3. Upcasting child হচ্ছে Parent

animal
↑
dog

4. Down casting (Data loss হয়)

Parent হচ্ছে child

animal
↓
dog

a = | 1 | 2 | 3 |
b ↗

a[0] এবং b[0] একই

Reference

Java তে array হচ্ছে Reference

a = | 9 | 81 |
b = | 9 | 8 |

a[1] এবং b[1] same না
Reference না
Java তে এভাবে কাজ করে

Multidimensional array can be jagged

int[][] data = { {1,2},
{3,4,5},
{6} }

4. Automatic initialization

```
int[] x = new int[3];
```

Declaration

```
int[] sampleArray;
sampleArray = new int[10];
```

OR

```
int[] sampleArray = new int[10];
```

↑ প্রত্যেক পজিশনে 0 থাকবে মোট (10টা 0)

Initialization

- During declaration

```
int[] sampleArray = {1,2,3,4,5};
int[] sampleArray = new int[] {1,2,3,4,5};
```

After declaration

```
int[] array;
array = new int[]{1,2,3,4,5};
```

```
array = {1,2,3,4,5} //compiler error
```

for loop 的 new style

```
int[] nums = {1, 2, 3, 4, 5, 6, 7}
int sum = 0;
for (int x : nums)
    sum += x;
```

Advantage: Avoid boundary error

Break

Lebeled Jump

```
Outer:
for (int i = 0; i < 5; i++) {
    break Outer;
}
```

09 August 2026

next মঙ্গলবার CT

Class – blueprite
Object – Instance of a class

Class এর Property হচ্ছে
1. Instance variable (fields/properties/attributes)
2. Instance method (Action/behavior)

```
class classname{
    datatype instance-variable1;
    datatype instance-variable2;
    datatype instance-variableN;

    returntype methodname1(parameter-list){
        //body of method
    }

    returntype methodname2(parameter-list){
        //body of method

    }
    returntype methodnameN(parameter-list){
        //body of method

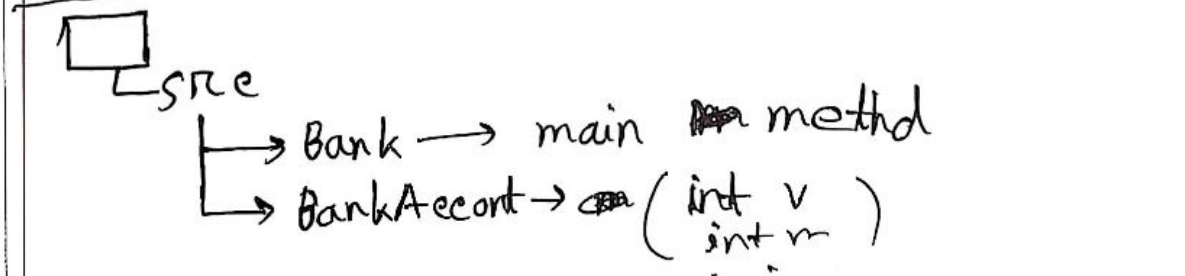
    }
}
```

public নিয়মানুযায়ী সমস্যা নাই

```
class Student{
    String name;
    int age;
    String id;
    void Details() {
        System.out.println("Name :" + name);
        System.out.println("Age :" + age);
    }
    int get_age() {
        return age;
    }
    void set_id(String i) {
        id = i;
    }
}
```

```
public class BankAccount {
    public String name;
    public String id;
    public double balance;

    public void deposit(double amount) {
        balance = balance + amount;
    }
    public void withdraw(double amount) {
        if (amount < balance)
            balance -= amount;
    }
}
```



creating objects —

ClassName refVariable = new ClassName(parameter-list)

Scanner scan = new Scanner(System.in)

BankAccount b1 = new BankAccount()

ET ও Class create করা হয়

9 August 2026

Write a program to create a class called Book.
The class should have the following attributes:

title
author
price

Create a method called displayInfo() that display the book's title author and price

```
public class Book {
    public String title;
    public String author;
    public float price;
    public void displayInfo() {
        System.out.println("Title of book: " + title);
        System.out.println("Author of book: " + author);
        System.out.println("Price of book: " + price);
    }
}
```

increasePrice(double p)=

```
void increasePrice(double p) {
    price = price + p;
}
```

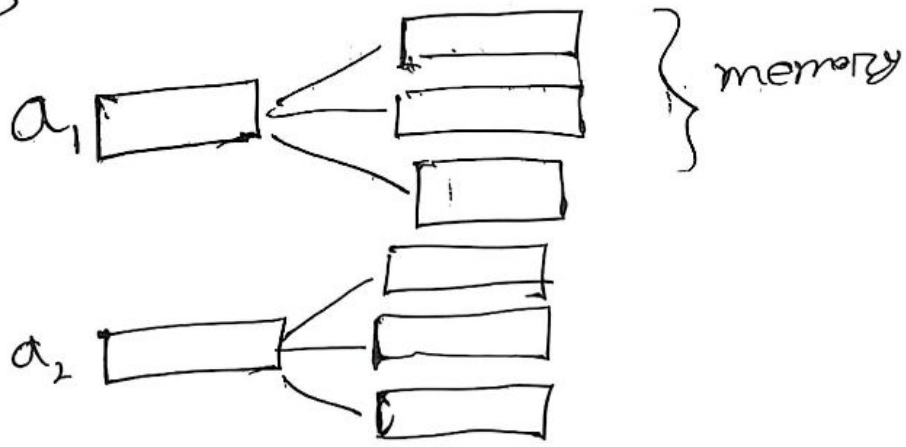
CT1 mandatory

Main.Java

```
public class Main {
    public static void main(String ar[]) {
        Book b1 = new Book();
        b1.title = "ABC";
        b1.author = "XYZ";
        b1.price = 150.00;

        b1.increasePrice(500.00);

        Book b2 = new Book()
        b2.displayInfo();
    }
}
```



reference variable

Create object and access members

Initializing fields and instance variable

3. Constructure (if default constructor use 2+1)

Constructor

```
import java.util.Random;
public class BankAccount{
    public String Name;
    public String Id
    public double balace

    //constructor without parameter

    public BankAccount(){
        id = new Random().nextInt(9999)+"";
        //name and balace will get default value;
    }

    //constructo with parameter
    public BankAccount(String _name, String _id, double _balance)
    {
        name = _name;
        id = _id;
        balance = _balance;
    }
```

main function ও call করার সময়-

```
BankAccount account = new BankAccount();
BankAccount account = new BankAccount("ABC","01",500);
```

"THIS" keyword

this use ~~করা~~ best practice

~~public class Main~~

~~{~~

~~public static void main()~~

~~{~~ string id;

double balance; String name;

```
public BankAccount(String name, String id, double balance)
{
    this.name = name;
    this.id = id;
    this.balance = balance;
}
```

1. Problem solving with Java
2. Class and object

16/08/2026

Scope and lifetyme of variable.

```
int main(){

    return 0;
}
```

main function এর local scope

```
int main() {
    if(cond){
    }
}
```

If এর local scope

```
int main(){
    for(int i=0; i<10; i++){
        int sum=0;
        sum+=i;
    }printf("%d", sum);
}
```

output: error

```
int main(){
    int sum=0, i;
    for(i=0; i<10; i++){
        sum+=i;
    }printf("%d", i);
}
```

for এর local scope

Scope ~~২ প্রকার~~ — Local Scope
Global Scope

```
public class Main{
    public static void main(String[] arg){
        int a=10;
        if(a=10){
            int x=10;
        }
        if(a==5){
            int c=5;
        }
        System.out.println(c);
    }
}
```

Output: Error

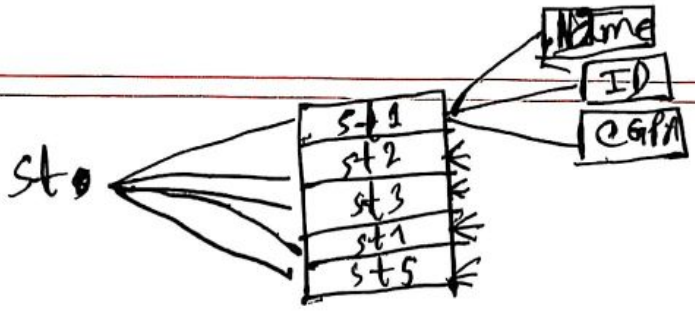
Arrays

Object array

```
int[] arr = new int[5]
Student object ka
Student arr = new Student[10]
Car arr = new Car[10];
```

(Object NULL nahi fill up karta initially)

~~for(int ; ;~~
~~student[i] = s;~~



~~Access~~
st[4].name;
st[1].id;

Student[] students = new Student[3] ← array এর new. Object এর না
students[0] = new Student("Rashid", 01115301", 3.8f, 50);
students[1] = new Student("Zaman", "01115302", 3.0f, 50);
student[2] = new Student("Lubna", "01115303", 3.5f, 50);

Student s = new Studen(. . .);
student[0] = s;

Parameter Passing

- 2 different ways
  - pass By value
  - Pass By Refernce

```
main() {
    int i = 10, j = 20;
    cout << i << " " << j << endl;
    swap(i, j);
    cout << i << " " << j << endl;
}
void swap(int &num1, int &num2){
    int temp = num1;
    num1 = num2;
    num2 = temp
}
```

output

10 20
20 10

```
main() {
    int i=10, j=20;
    cout — ————————
    — —— ——— ———
    ——— ——
    —
}
void swap (int num1, int num2) {
    int temp = num1;
    num1 = num2;
    num2 = temp;
}
```

output

10 20

10 20

(আসলে এর address নিয়ে কাজ করেছিল)

Object pass করলে & মূল্য লাগে না।
Object নিজেই address hold করে।

```
public clas passBylalue{
    public static void main(String[] args){
    Test t = new Test("CT1", 10);
    t.display()
    updateScore(t, 15.0f)
     System.out.println("After updat ");
      t.display();
}  void updateScore(Test test, float newScore){
        test.score = newScore;
}  }
```

next for garbage Collection

Design a class called Animal

* Instance variable:
  no_of_leg (int), has_tail (boolean), breed (String)
* Write a parameterize constructor.
* Instande methodi
  - display_info()
  - sound()

```
public class Animal(){
    int no_of_legs; boolean has_tail;
    String breed;
    Animal(int legs, boolean tail, String breed){
        this.no_of_legs = legs;
        this.has_tail = tail;
        this.breed = breed;
    }
    public void display_info(){
        System.out.printf("No of legs = %d\n", no_of_legs);
        System.out.println("Has Tail = "+has_tail);
        System.out.println("breed = "+this.breed);
    }
    public void sound(){
        System.out.println("Meaw");
    }
}
```

```
public class Product {
    String productName;
    double pricePerUnit;
    int quantityInStock;

    Product(String Name, double unit, int q) {
        this.productName = Name;
        this.pricePerUnit = unit;
        this.quantityInStock = q;
    }
    public void sell(int units) {
        if (this.quantityInStock >= units)
            this.quantityInStock -= units;
        else
            System.out.println("Sale can't be completed");
    }
    public void restock(int units) {
        this.quantityInStock += units;
    }
    public double totalValue() {
        return this.pricePerUnit * this.quantityInStock;
    }
}
```

Very, Very Good

30/8/2026

Static

instance variable
static variable
class variable

Score
- Team
- Individual

```
class Player{
    int p_score;
    static int t_score;
    Player(p_score){
        this.p_score = p_score; this.t_score += p_score;
    }
    void hit_six(){
        this.p_score += 6;
        this.t_score += 6;
    }void display(){
        System.out.println("Team Score: " + this.t_score);
        System.out.println("Player Score: " + this.p_score);
    }
public class Main{
    public static void main(String[] args){
        Player P1 = new Player(20);
        P1.hit_six();
        P1.displayInfo();
        Player P2 = new Player(10);
        P2.displayInfo();
    }}
```

static ত্ত্ব সমস্ত object এ same value থাকে।

direct class এর error call করা যায়।

Player.t_score = 66;

instance method
static method

Static Method ও Non-static মিলিয়ে use করতে পারব না। error দিবে

Insid previous Player class

```
static void hit-four(){
    new-method();
}
static void new-method(){

}
```

Non-static থেকে static use করতে পারব না।

Write a Java progrram.. class Studem

Each student has.

- name
- id
- marks
- the total number of student created

Creat constructor

display method

In main() three object

display each studet

display total number of student Created

```
public class Student {
    String name;
    String id;
    int marks;
    static int total;

    Student(String name, String id, int marks)
    {
        this.name = name;
        this.id = id;
        this.marks = marks;
        this.total += 1;
    }
    void display() {
        System.out.println("Name= " + this.name
                + "\nID = " + this.id
                + "\n Marks = " + this.marks);
    }
}

public class Main()
    public static void main(String[] cats){
        Student st1 = new Student("Utsob", "3", 99);
        Student st2 = new Student("Shohan", "9", 100);
        Student st3 = new Student("Zabed" "52", 80);
        st1.display(); st2.display(); st3.display();
        System.out.println("Total student = " + Student.total);
    }
}
```

ে৮ ত্ত ২য়ে Inheritance শেখা

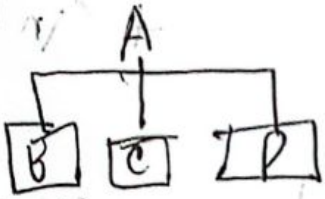
OOP Principles

Inheritance (উত্তরাধিকার)

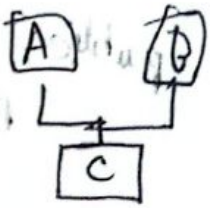
Parent class এর গুণ (variable, method)
child class পায়।

Types of Inheritance

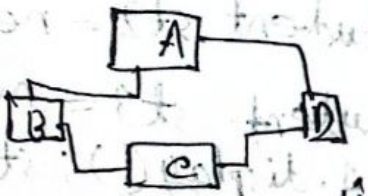
- single — 1 parent — A, 1 child — B
- Multi level — Gurandparent — A, child — B, grandchild — C
  - A এর অধিনে B আছে
  - B " " C এর
  - A " " C "
- hierarchical inheritance



- Multiple — Multiple parent, 1 child



- Hibrid



Java doesn't support mutlple inheritance

2/9/26

Exercise

এখন সবার মনোভাবটি, মুলতে পারি নাই।

```
public class Student {
    String name; String id; int marks;

    Student(String name, String id, int marks)
    {
        this.name = name;
        this.id = id;
        this.marks = marks;
    }
}
import java.util.*;
public class Main{
    public static void main(String[]args) {
        Student [] student = new Student[3];
        Scanner scan = new Scanner(System.in);
        for (int i=0; i<3; i++ ){
            System.out.println("Enter information of student"+i);
            System.out.println("Name:");
            String name = scan.nextLine();
            String Id = scan.nextLine();
System.out.println("Id:");
System.out.println("Mark");int marks = scan.nextInt();
            student[i] = new Student(name, Id, marks);
            student[i] = s;
        }
```

```
for(int i=0; i<3; i++)
{
    System.out.println("Student"+i+"Information"
            +"\nName = "+student[i].name
            +"\nID = "+student[i].id
            +"\nMarks = "+student[i].marks);

}
}
}
```

```
for (Studen a: s)   // student array
{
    if(a != null)
        System.out.println(a.name          );

{
```

Inheritance @ Syntax

```
class Parent{

}
class Child extends Parent {

}
```

keyword

```
class Animal{ }
class Dog extends Animal { }
class Cat extends Animal{ }
```

Superclass — parent class
Subclass — child class

```
class Vehicle{
    Vehicle(){
        System.out.println("This is a vehicle");
    }
}
class Car extends vehicle{
    Car(){ super();
        System.out.println("This vehicle is Car");
    }
}
public
```

```
public clas Test {
    public static void mai(String [] args){
        Car obj = new Car();
    }
}
```

Output

This is a Vehicle ← এটি Automatic এসেছে ~~Constructor এ এসেছে~~.
This Vehicle is Car

Vehicle এর constractor এ parameter থাকায় super keyword লাগবে super দিয়ে parent class এর contractor call করতে। (Parameter থাকলে super ও লাগবেই)

5/9/2026

```
Class Car { }
public class Main {
    public static void main (String[]args) {
        Car c = new Car();
        System.out.println (c);
    }
}
```

output

@ 1bd16a6

default toString method address return করে

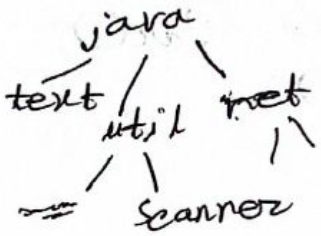
```
class Car {
    public String toString() {
        return "This is from Car class";
    }
    public void method() {
        System.out.println("This is a method");
    }
}
public class Main {
    public static void main (String[] args) {
    Car c = new Car();
    System.out.println(c); ← toString মা দিলেও কাজ তা
                              দিলে করা
    c.method(); ← method call করে Print করা!
}
```

Another example of toString()

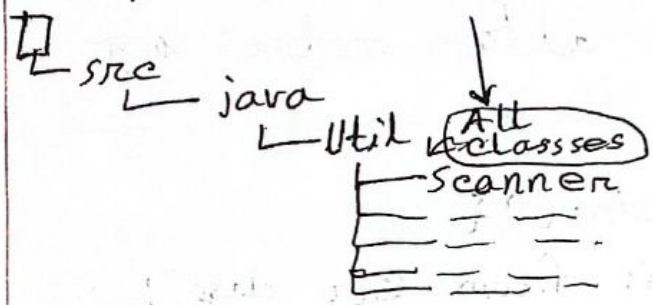
```
int a = 5;
int b = 6
public String toString() {
    return this.a + " " +this.b;
}
```

5 6

Package (বিস্তারিত এবং important)
oop lecture3_classobject.pdf
slide 68



import java.util.*;



CT তে Object এর Array ছিল
Inheritance থেকে simple থেকে
Moderate question
Package থেকে না
8/9/2026 CT-2

WAP
- Administration
- Education
  - CSE
  - Civil
  - EEE
    - main.java
    - myclass

```
package wap.cse;
public class Test {
    ---  ---  ---
}
import wap.cse.Test;
public class TestMain {
    ---  ---  ---
}
```

Package use করলে Public লাগবে
নাহলে error আসতে পারে

hierarchial inheritance

```
public class LibraryItem {
    public String title;
    public double price;
    public LibraryItem(String title, double price){
        this.title = title;
        this.price = price;
    }
    public String toString() {
        return "Title: " + this.title + "Price: " + this.price;
    }
}
public class Book extends LibraryItem {
    public String author;
    public Book (String author, String title, double price){
        super(title, price);
        this.author = a;

    }
    public String toString(){
        return "Authror: " + this.author + " " + super.toString();

    }
}
public class Magazine extends LibraryItem{
    int issueNumber;
    public Magazine(int issueNumber, String title, double price){
        super(title, price);
        this.issueNumber = issueNumber;
    public String toString(){
        return "Issue Number: " + this.issueNumber + " " + super.toString();
    }
}
public class Main {
    public static void main(String[] args){
        Book book = new Book("cat", "Rice", 50);
        Magazine mag = new Magazine(5, "Rice", 50);
        System.out.println(book + mag);
    }
}
```

```
class Library{
    string name;
}
class Book extend Library{
    Book (string name){
        this.name = name;
    }
}
```

this দিয়ে parent এর veriable call করা যায় যখন parent এর constructor এ parameter নাই।

6 september

Static আর tostring ও থাকতে পারে।

CT2 object ও array

Array থেকে কিভাবে শুরু থাকবে Inheritance থাকবে

Mid এ সবকিছু হয় না যা যা পড়াইছে থাকবে output দিবে কিভাবে output আসবে তা ও দিবে।

22/9/2026

Encapsulation

Lecture_3_4_class Object.pdf

Page 82

Access Control / Access Modifier

4 Type

- Public
- Protected
- Default / Package access — No modifier
- Private

Private

Members declared private are accessible only in the class itself

- sub-class
- non-sub class
- same package
- different package

} কোনটাতেই Private use করা যায় না

Different class থেকে public method কে call করলে সে private কে Access করতে পারবে বা ডাইরেক্টলি Different class private কে access করতে পারে না।

Example: private String name;

Default / Package Access

- Example: ~~Public~~ String name;

Sub class ✓ ⟶ Child class হলেও
non-sub class ✓ ~~✗~~ Different package
same package ✓ এ আমরা access করতে
diff package ✗ পারি না।

```
package test;
class Default{
    String name = "Default";
}
```

Class under different package

```
package test1;
class DefaultExample{
    public static void main(String[] args){
        Default d = new Default();
        System.out.println(d.name); //Compile error
    }
}
```

class under same package

```
package test;
class DefaultExample1{
    pub ~~~~~ (string[] ~~)
        Default d = ne ~~~~ ()
        ~~print~~(d.name) //OK
    }
}
```

Protected

~~Members decl~~

Sub - class ✓
non-sub class ✗

same package ✓
diff package ✗ ⟶ Sub-class 2 ways Ok ✓

Public

Sub-class ✓
non-sub class ✓
same package ✓
different page ✓

Incapsulation ❓ Concept - getter, setter method

```
class Human {
    int age;          // private
    public String name;
    private int age;
    public void set_age(int x){
        if(x>=0 && x<=100) age=x;
    }
    public String getName(){
        return this.name;}
    public int getAge(){
        return this.age;}
}
class Main{
    public static void main(String[] args){
        Human h=new Human()
        h.age=50;// error
    }
}
```

CGPA

Example Incapsulation code

```
class Human {
    private int age;
    private String name;
    public int getAge() {
        return this.age;
    }
    public void setAge(int x) {
        if(age >=0 && x <=100) this.age=x;
    }
    public String getName() {
        return this.name;
    }
    public void SetName(String x) {
        this.name = x;
    }
}
```

Private member child class ও সরাসরি access করা যায় না।

Private variable নিয়ন্ত্রণ করে get, set method নিয়ন্ত্রণ করা যায় না।

27/9/26

```
class Demo{
    private int age;
    public Demo(int age) {
        ~~this.age=age;~~ setAge(age);
    }
    public void setAge(int age){
        if(age>18 && age.<=36) this.age = age;
    }
}
```

Constructor ও Method use করা

Problem 1

Create a class Employee with private name, age, salary, setters must reject an empty name an age outside 18-65, and a negative salary. Provide getters and test with valid and invalid values.

Problem 2

Create a class ATM with:

- cardNumber : read only. Set only through the constructor, getter only (no setter)
- pin : write only. Provide setPin(int) (Must be a 4 digit Number, 1000 to 9999) and checkPin(int) returning boolean, but no getPin()

```
class Employee{
    private String name;
    private int age;
    private double salary;
    public void setName(String name){
        if(name != NULL) this.name=name;
    }
    public String getName(){
        return this.name;
    }
    public void setAge(int age){
        if(age>=18 && age<=65) this.age=age;
    }
    public int getAge(){
        return this.age;
    }
    public void setSalary(double salary){
        if(salary>=0) this.salary=salary;
    }
    public double getSalary(){
        return this.salary;
    }

    public Employee(String n, int a, double s){
        setName(n); setAge(a); setSalary(s);
    }
}
```

29/9/2026 Sunday তে CT3 আর CT1 এর alternative

oop
- Inheritance
- Encapsulation
- polymorphism

Polymorphism

oopLecture6 – oopPrinciples.pdf

3 Types
- Method overriding
- Method overloading
- Subclass polymorphism

→ Method signature (name and argument list) and return type must be same

```
class Animal {
    void display() { syso("Animal"); }
}
class Dog extends Animal {
    void display() { syso("Dog"); }
}
```

↑ কোন extra argument বা return type ভিন্ন হলে override হবে না

Method overloading

Same name with different signature

```
class Animal {
    String name;
    Animal() { System.out.println("C-1"); }
    Animal(String name) { this.name = name; System.out.println("C-2"); }
}
```

method overloading

```
void method1(){
        println("M-1");
}
void method2(String n){
        println("M-2");
}
void method1(int n){
        println("M-3");
}
}
```

// String return করে এমন কিছুতে সমস্যা না কারণ return type overiding (না)।

```
class Box{
double width, hieigth depth
Box(double w, double h, double d){
    width = w;
    height = h;
    depth = d;
}
Box(doule n){
    this(n, n, n);
}
}
```

Subclass polymorphism

```
class Animal{
    string weigh, color;
    void eat(){
        syso("Form A-E");
    }
    void sleep(){
        syso("Form A-S");
    }
}
class Bird extends Animal{
    void canFly(){
        syso("Birds can Fly");
    }
    void sleep(){syso("From B-S");
}

public class Main
{ public static void main(------){
        Animal a = new Animal();
        Bird b = new Bird();
        b.canFly();
Upcasting → Animal c = new Bird(); // Type casting
        Animal d = b;
        c.eat(); ⟶ // A-E
        c.sleep(); → // B-S
        c.canfly(); → complier error
```

Cat, dog class কোথাও সরাসরি আমরা Animal কে child আমি তুলি নাই।

```
public class Main{
    static void makeAnimalSound(Animal animal){
        animal.makeSound();
    }
    public static void main(---){
        Animal a1 = new Dog();
        Animal a2 = new Cat();
        a1.makeSound();
        a2.makeSound();
    }
}
```

আর্থাৎ আমরাকিন্তু কোথাও;

04/10/2026

```
class Animal{
    void eat(){
        System.out.println("A-E");
    }
}
class Dog extends Animal{
    void eat(){
        System.out.println("D-E");
    }
    void sleep() { --- print("D-S"); }
}
public class Main
{
    public static void main(String[] args) {
        Animal a = new Animal();
        Dog d = new Dog();
        a.eat(); // A-E
        d.sleep(); // D-S
        Animal b = new Dog(); // upcasting
        b.eat(); // D-E
        Animal c = new Cat();
        c.eat(); //A-E
        b.sleep(); // compile time error (Parent에 sleep() 없음)
    }
}
class Cat extends Animal{
    void eat(){ print("
}
```

Alternative main class — subclass polymorphism & repetition

```
public class Main
{ static void callMethods(Dog a){
        a.eat();
  }
  static void callMethods(Cat a){
        a.eat();
  }
```

// সবগুলো ক্লাস এর জন্য আলাদা আলাদা লিখতে হচ্ছে

```
static void callMethod(Animal a){
        a.sleep();
}
```

এই একটি method দিয়ে সব Subclass এর কাজ করা যাবে যদি upcasting করা হয়

```
Main method{
    Animal b = new dog();
    callMethod(b); // compile time error
}
```

```
static void callMethod(Animal a){
    Dog x = (Dog) a; // Downcasting
    x.sleep();
}
```

```
Main method{
    callmethod(b); // D-S
}
```

কারণ animal এ sleep ছিল না তাই error দিল এখানে downcasting করে Animal কে Dog এ convert করা হয়েছে তাই Dog এর method টি call হয়েছে আর error দেয়নি।

cat class

```
class Cat extends Animal{
    void run{ --print("C-R"); }
```

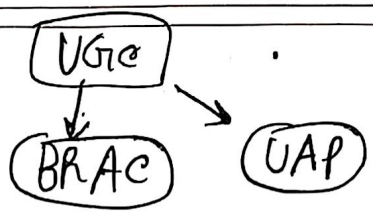
```
static void callMethod (Animal a) {
    if (a instanceof Dog) {
            Dog d = (Dog) a;
            d.sleep(); //D-S
    }
    else if (a instanceof Cat) {
            Cat c = (Cat) a;
            c.run(); //C-R
    }
    else a.eat(); //A-E
}
```

instanceof use করে কোন class এর object তা check করা যায়

UGC - University grand comission

- Static Binding
  - Compile time

- Dynamic Binding
  - Runtime



Abstact class

Abstract class থেকে object তৈরি করা যায় না

```
public abstract class NameofClass {
    // Any number of instance variable, constructor, concrete methods
    // Zero or more abstract method declarations
}
```

Abstract ছাড়া class এ কোন Method এ body নাই

abstract class এ abstract method child class follow করে

(child class must override those abstract methods

Animal কে abstract class

```
abstract class Animal {
    String name="a";
    double weight;
    // constractor বসানো দরকার নাই
    void eat() { System.out.println("A-E"); }
    abstract void sleep(); // Child class এখানে
                          এই Method বসানোবাধ্যতামূলক
}
class Dog extends Animal{
    void eat() { System.out.println("D-E"); }
    void sleep(){ System.out.println("D-S"); } // sleep না বসালে
                                                error আসবে parent
                                                এ abstract method

Main{
public static void main(String[] args){
    Dog d = new Dog();
    print(d.name());
    print(d.name()); // a
```

18 Oct

Framework

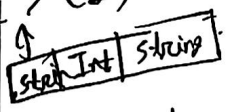
25/08/2026

Lecture5- Generic Class- Collections.pdf

```
List<String> list1 = new ArrayList<>;
Student s = new Student("Anamika", 1, 3.95);
Student s1 = new Student("Anik", 2);
```

নতুন ক্লাস Design নাহয়ে না। নতুন Constructor লাগবে

List<> l1;




```
s.display();
```

Generic Class — সাধারণত Capital letter

```
class Gen <T> {
    T ob;
    Gen(T o) { ob = o; }
    T getOb() { return ob; }
    void showType() {
        System.out.println("Type of T: "+ob.getClass().getName());
    }
}
```

Slide 3

```
public static void main(String[] args){
    Gen<Integer> iOb = new Gen<Integer>(88);
    iOb.showType();
    int v= iOb.getOb();
    System.out.println("value: "+v);

    Gen<String> strOb = new Gen<String>("Test");
    strOb.showType();
```

Slide 4

Repetition of datatype can be avoided in 2 ways

Theory

(a) Type inference

```
Gen<Integer> iOb = new Gen<>(98);
```

(b) local variable type inference

```
var iOb = new Gen<Integer>(88)
```

Slide 5

Slide 6

Slide 7

boundary

<T extends Number>

Slide 8

Wild Card

? দিলে যেকোন Datatype

Slide 9

~~Bounded Wildcard~~

27/10/26

Sunday CT 5

generic class (slide 1-8)

Slide 9

Wildcard

Slide 10

Bounded wildcard

Try - catch

2/3/26

~~try catch~~